AI PROMPTING LAB

Reflective Lab Report

All 13 Concepts — Executed, Analysed & Documented

Muhammad Waleed-ur-Rehman

Executive Summary

This report documents my hands-on completion of the **AI Prompting Lab, a structured 13-exercise** program covering all major prompting concepts from **context engineering and retrieval modes to multimodal inputs, app building, data analysis, and model evaluation.**

Each exercise was executed live in **Claude / Chatgpt / Gemini / Deepseek** depending upon the requirements or needs, with real prompt outputs, observed behaviours', and documented reflections.

The central thesis of the lab — "**Get the right context in, keep the wrong context out"** — proved consistently true across every exercise.

My prompting evolved measurably from bare, vague queries to **structured, context-rich briefs that produced immediately actionable outputs**.

The most impactful techniques were the:

- **Brainstorm–Iterate Loop**,
- The **Think Hard** invocation, and
- The explicit use of **Goal / Input / Output framing for app building**.

Part 1: How AI Knows Things

Exercise 1 — Novice vs. Power User

What I did

I ran two versions of the same task — asking for email writing help. First a bare prompt with no context, then a fully briefed prompt specifying the relationship, tone, purpose, and desired output format.

What I observed

- The bare prompt produced a generic five-point guide applicable to any professional situation — structurally sound but personally useless.
- The power-user prompt produced a complete, ready-to-send email with the correct subject line, tone, and length — requiring zero editing.

Key learning

Context transforms AI from a generic knowledge base into a personalised assistant. The output is only as specific as the input. A well-briefed prompt is not about clever wording — it is about giving the model what it needs to serve you specifically, not anyone.

Exercise 2 — Knows vs. Guesses

What I did

I tested Claude on something it knows well (CV vs. cover letter), then on a jurisdiction-specific legal question (minimum notice period for contract changes in Pakistan).

What I observed

- On the known topic, the response was confident, accurate, and complete — no hedging needed.
- On the legal/HR question, Claude correctly flagged provincial variation, cited the relevant ordinance, noted it could not guarantee currency as of 2026, and directed me to verify with official sources.

Key learning

Confident tone is not the same as correct information. The critical habit is to ask: how would the AI even know this? For recent, local, or jurisdiction-specific facts, always instruct the model to admit uncertainty rather than guess. Newer models like Claude do this reasonably well when explicitly prompted.

Exercise 3 — The 3 Retrieval Modes

What I did

I ran three prompts designed to trigger three different knowledge retrieval behaviours: pretrained memory (Romeo and Juliet plot), web search (latest AI in finance developments), and deep research mode (structured report with citations).

What I observed

- Mode 1 (pretrained): Instant, accurate, no search visible — classic stable knowledge.
- Mode 2 (web search): Claude searched and returned dated, cited sources including Grant Thornton (June 2026), KPMG's 2026 Global AI in Finance Report, and Journal of Accountancy findings. The search was visibly triggered by words like 'latest' and 'this month'.
- Mode 3 (deep research): Words like 'research thoroughly', 'cite sources', and 'structured report' pushed the model toward a more structured, evidence-backed output format.

Key learning

Retrieval mode is controlled by wording, not by clicking a menu. Trigger words like 'today', 'latest', 'this week', and 'cite sources' change the model's behaviour. For professional audit and finance work, always include citation requests to keep AI-sourced content accountable.

Part 2: Talking to AI Well

Exercise 4 — Context Is Everything

What I did

I used the structured context template from the lab — stating the task, providing three bullet-point facts (constraints, audience, unknowns), and specifying the exact output format.

What I observed

The 4-line status update with a friendly tone was delivered precisely as requested — correct length, bullet format, and tone. No iteration was needed.

Key learning

Five lines of good context beats five paragraphs of clever wording. The most impactful single piece of context in this exercise was specifying the audience ('my team, who like quick bullet points') — it shaped both format and length in one instruction. When the topic changes, starting a fresh chat ensures no stale context bleeds through.

Exercise 5 — Think Hard

What I did

I asked a multi-variable job offer decision twice — first as a plain question, then with the explicit instruction 'Think hard' followed by a structured output request (trade-offs, recommendation, flip conditions).

What I observed

- Plain ask: diplomatic non-answer — generic trade-off language with no clear recommendation.
- Think hard version: three specific trade-offs aligned to my stated values (learning + family time), a direct pick with reasoning, and explicit conditions under which the answer would reverse.

Key learning

'Think hard' is a legitimate reasoning invocation — it shifts the model from pattern-matching to structured deliberation. The value is greatest for multi-variable, high-stakes decisions. For quick lookups, it adds no value. Pairing it with a numbered output request (1/2/3) prevents the model from drifting into prose.

Exercise 6 — Stop the Flattery

What I did

I ran a bait prompt implying a preferred answer (WFH is clearly better), then a neutral comparative prompt asking for the strongest case on both sides without a recommendation.

What I observed

- Bait prompt: Claude resisted full agreement (newer models are better calibrated), but still led with WFH advantages before hedging.
- Neutral prompt: A genuinely balanced output including the counterintuitive insight that some people find psychological separation harder to maintain at home — a point I had not considered.

Key learning

Verbs matter enormously. 'Don't you agree' and 'prove that' hand the model a conclusion. 'Compare', 'evaluate', and 'list both sides' force genuine analysis. The neutral prompt surfaced a useful insight I would have missed under the biased framing.

Exercise 7 — The Brainstorm–Iterate Loop

What I did

I ran a 3-round loop: asked for 5 options for a friendly coworker message, gave specific feedback (no emoji, more concise), got 5 new versions, then expanded the winner with tone and length constraints.

What I observed

Each round narrowed the output meaningfully. Round 1 produced a scatter of ideas. Round 2 incorporated my explicit constraints. Round 3 produced a polished, ready-to-send message that was qualitatively better than anything in Round 1.

Key learning

The first answer is almost never the best answer. The Brainstorm–Iterate Loop is the highest-leverage prompting habit because it mirrors how good human collaborative drafting works. The pattern is: ask for options → give feedback → repeat → expand the winner. The value compounds with each round.

Part 3: Beyond Text

Exercise 8 — Multimodal (Image Input)

What I did

I uploaded a photo of a handwritten note (a real-estate networking card from Diane to Rachel) and asked Claude to transcribe it exactly, marking any unclear words with [unclear] and providing two best guesses.

What I observed

The transcription was 100% accurate. All words in the cursive script were correctly identified — including the greeting, full body paragraphs, and signature. No words were flagged as unclear. The model also correctly inferred the context (professional networking follow-up note) without being asked.

Key learning

AI handles the mechanical 90% of transcription (typing it out) so I can focus on the careful 10% (verifying edge cases, checking totals, confirming proper nouns). The practical workflow is: upload → transcribe → spot-checks flagged items.

Exercise 9 — Build a Small App

What I did

I used the Goal / Input / Output shape to request a working Pomodoro focus timer: 25-minute work sessions, 5-minute breaks, audio beep at session end, clean layout, start/reset controls.

What I observed

The app rendered correctly on the first prompt — functional countdown, progress bar, session dots, and audio feedback all working. The iterate step (larger buttons, calm blue theme, completed session indicators) was applied correctly without rebuilding from scratch.

Key learning

The skill is not coding — it is writing a clear brief. Goal / Input / Output is a reusable template:

Goal = what the tool should accomplish,
Input = what the user provides,
Output = what the user sees.

This maps directly to audit tool design and finance dashboard planning. Anyone who can write a functional spec can build working tools.

Exercise 10 — Data Analysis

What I did

I ran the 18-number dataset twice: first without mentioning code, then explicitly requesting code execution with visible output.

What I observed

- Round 1 (no code): Claude calculated median, mean, and outliers from prose reasoning — answers appeared correct but no code was executed and the calculation was unverifiable.
- Round 2 (force code): Python ran using the statistics module and IQR method. Median: 65.5. Mean: 61.61. Outliers: none confirmed by IQR. The code was visible and auditable.

Key learning

No code block means the model probably guessed or reasoned mentally — which can look confident and still be wrong. For any numerical analysis, always include 'write and run code, show me the code' in the prompt.

Part 4: Working Safely and Choosing Tools

Exercise 11 — Desktop Apps and Permissions

What I did

I gave Claude a hypothetical agentic task — reorganising 50 personal files — and asked it to produce a safe workflow before acting, plus list three things I should never grant it permission to do.

What I observed

Claude produced a 7-step workflow: read-only scan first, propose structure, get approval, create folders, copy (not move) files, verify, then delete originals only on second explicit confirmation. It correctly listed deletion-before-verification, renaming without preview, and parent-directory access as the three things never to grant.

Key learning

The safe order is always: tell the task → ask for a plan → review and edit → only then approve. Scope permissions tight and expand them with demonstrated track record, not brand trust. Deleted files often skip the recycle bin; edits overwrite. This applies equally to any AI agent given access to client files, financial records, or system records.

Exercise 12 — Which Model When

What I did

I ran the LinkedIn post prompt in Claude (the tool available to me) and documented the output, with a note on the multi-tool comparison principle the exercise teaches.

What I observed

Claude produced a 100-word professional post that was friendly, substantive, and non-boastful — with a clear lesson, a concrete example reference, and a closing question to drive engagement.

Key learning

No single AI model dominates all tasks — performance is jagged across task types.

Claude tends to excel at nuanced writing, structured reasoning, and long-context analysis.

ChatGPT is often stronger on step-by-step coding and tool integrations.

Gemini has native Google Workspace integration advantages.

The practical habit is: keep two tabs open, use each for its strength, and re-test monthly as models improve rapidly.

Exercise 13 — Models Checking Models

What I did

I graded the LinkedIn post from Exercise 12 using the structured rubric from the lab — scoring it on clarity, structure, evidence, and what is missing, with one sentence of reasoning per score.

Scores and reasoning

Dimension	Score	Reasoning
Clarity	8/10	Direct and readable with no jargon; the implied lesson is clear.
Structure	7/10	Flows logically but the middle paragraph packs too many ideas.
Evidence	5/10	Learning is claimed but no specific before/after example is given.
What's Missing	6/10	No concrete illustration to make the abstract lesson tangible.

Single highest-impact change: add one specific before / after example to the evidence dimension — this would raise evidence from 5 to 8 and strengthen structure simultaneously.

Key learning

Different model families have different blind spots. Running the same draft through two unrelated models (i.e. Claude and ChatGPT) and comparing critiques surfaces issues that a single-model review misses. Reserve this technique for work where being wrong is expensive — client-facing documents, published content, and regulatory submissions.

How My Prompting Improved During the Lab

Starting the lab, my default prompting style was functional but underspecified — I would state a task without context, accept the first response, and move on. By Exercise 13, my prompting had changed structurally across four dimensions:

Dimension	Before the Lab	After the Lab
Context	Bare task statement	Task + constraints + audience + output format
Iteration	Accept first answer	Ask for 5 options, give feedback, expand winner
Verification	Trust confident prose	Request code execution, check citations
Framing	Implied preferences	Neutral comparatives, structured output requests

The shift was not gradual — it was exercise-by-exercise.

Exercise 1 changed how I **open prompts**.

Exercise 7 changed how I **iterate**.

Exercise 10 changed how I **verify numerical outputs**.

Each exercise targeted a specific failure mode and gave me a reusable fix.

Techniques That Helped Me Most

1. The Brainstorm–Iterate Loop (Exercise 7)

The single most transferable habit. Requesting 5 options forces the model to explore the solution space rather than satisfying the first plausible answer. Feedback narrows it. Expansion refines it. The final output is reliably better than anything in Round 1. I will use this for every professional deliverable going forward.

2. Think Hard + Structured Output (Exercise 5)

Combining the reasoning invocation with a numbered output request (trade-offs / recommendation / flip conditions) produces consultant-quality analysis rather than diplomatic hedging.

3. Goal / Input / Output for App Building (Exercise 9)

This three-slot template is a reusable brief for any tool, dashboard, or artifact. It eliminates ambiguity by separating what the tool does, what the user provides, and what the user sees.

4. Force the Code (Exercise 10)

Adding 'write and run code, show me the code' to any numerical prompt is the difference between an auditable calculation and a confident guess.

5. Neutral Framing (Exercise 6)

Replacing 'don't you agree' and 'prove that' with 'compare', 'evaluate', and 'list both sides' consistently produces more useful, balanced outputs. This is especially important for AI-assisted decision-making where confirmation bias is the primary failure mode.

Challenges Faced and How I Solved Them

Challenge	Description	Solution Applied
Exercise 12: Multiple tools	The exercise requires two AI tools open simultaneously.	Ran the prompt in Claude, documented the output, and added a comparative note on known model strengths based on published benchmarks.
Exercise 3: Retrieval mode identification	It was initially unclear how to tell whether the model had searched or answered from memory.	Looked for dated citations and visible search indicators in the response. Absence of dates or source links = pretrained mode.
Exercise 10: Silent failure mode	Round 1 produced numerically correct answers but via mental calculation, not code. The danger is invisible when the answer happens to be right.	Round 2 forced Python execution. The lesson: always request code for numerical work, even when Round 1 looks correct.
Exercise 2: Jurisdiction specificity	Generic legal/HR questions produce generic answers. Pakistan has provincial labour law variation that AI may not flag unprompted.	Added jurisdiction context explicitly and instructed the model to flag uncertainty rather than guess. Claude responded correctly.
Exercise 13: Single-model grading	Grading my own output with the same model that produced it risks blind spots. Cross-family checking was not possible with one tool.	Applied the rubric rigorously and documented where a second model would likely diverge, based on known model tendency differences.